

Vue3组合式API：组件生命周期函数全解析

在Vue组件的整个存在周期中，会经历从创建、挂载、更新到卸载的一系列阶段，**生命周期函数（钩子函数）**便是在这些关键节点被触发的回调函数。它允许开发者在组件不同阶段注入自定义逻辑，是实现组件复杂行为的核心能力之一。Vue3组合式API通过 `onXxx` 系列函数重构了生命周期体系，相比选项式API更灵活、更易复用。本文将完整解析Vue3组合式API中生命周期函数的用法、特性及实战场景。

一、生命周期核心概念：组件的“成长轨迹”

组件的生命周期本质是Vue框架管理组件实例的一套流程，每个阶段对应组件状态的特定变化。理解生命周期的核心价值在于：

- 精准控制时机**：在组件渲染完成后初始化DOM操作，在数据更新前拦截处理，在组件卸载前清理资源（如定时器、事件监听），避免内存泄漏；
- 拆分复杂逻辑**：将不同阶段的业务逻辑分散到对应生命周期函数中，使代码结构更清晰；
- 适配组合式API**：与 `ref`、`reactive` 等API配合，实现数据与生命周期的联动管理。

Vue3组合式API彻底摒弃了选项式API中 `created`、`mounted` 等分散的选项，统一通过导入 `vue` 包中的钩子函数使用，所有生命周期逻辑可集中在 `<script setup>` 或 `setup()` 函数内。

二、Vue3组合式API生命周期函数总览

Vue3保留了Vue2核心的生命周期阶段，但在命名和使用方式上进行了优化，同时新增了适配组合式API的调试钩子。以下是组合式API中常用生命周期函数与选项式API的对应关系及功能说明：

组合式API钩子函数	对应选项式API	触发时机	核心用途
<code>onBeforeMount</code>	<code>beforeMount</code>	组件挂载到DOM前（模板已编译，DOM未生成）	初始化数据准备，载数据（非DOM挂载阶段）
<code>onMounted</code>	<code>mounted</code>	组件挂载完成（DOM已生成并插入页面）	执行DOM操作（如三方插件、绑定事件）
<code>onBeforeUpdate</code>	<code>beforeUpdate</code>	组件数据更新前（DOM尚未重新渲染）	获取更新前的DOM，保存滚动位置
<code>onUpdated</code>	<code>updated</code>	组件数据更新后（DOM已重新渲染）	基于新DOM执行第三方插件状态更新

<code>onBeforeUnmount</code>	<code>beforeDestroy</code>	组件卸载前（实例仍完整存在）	清理资源（清除事件、销毁插件实例）
<code>onUnmounted</code>	<code>destroyed</code>	组件卸载完成（实例被销毁，DOM已移除）	收尾工作，如通知状态
<code>onActivated</code>	<code>activated</code>	keep-alive包裹的组件被激活时	恢复组件状态，如数据
<code>onDeactivated</code>	<code>deactivated</code>	keep-alive包裹的组件被失活时	暂停组件行为，如
<code>onErrorCaptured</code>	<code>errorCaptured</code>	子组件抛出错误时（包括子组件的子组件）	捕获并处理错误，溃



注意：组合式API中没有对应选项式 `created` 和 `beforeCreate` 的钩子——因为 `<script setup>` 或 `setup()` 函数本身会在组件实例创建后、DOM挂载前执行，其内部代码可直接替代这两个钩子的功能。

三、基础用法：生命周期函数的调用方式

Vue3组合式API的生命周期函数需从 `vue` 包中导入后使用，所有钩子函数均在 `<script setup>`（或 `setup()`）内调用，且支持多次调用同一钩子（执行顺序与调用顺序一致）。

1. 基本使用示例（<script setup>）

最常用的生命周期组合：初始化数据（替代created）、挂载后操作DOM、卸载前清理资源。

Code block

```
1  <template>
2    <div class="lifecycle-demo">
3      <h3>生命周期演示</h3>
4      <p>当前计数: {{ count }}</p>
5      <button @click="count++">增加计数</button>
6    </div>
7  </template>
8
9  <script setup>
10  import { ref, onBeforeMount, onMounted, onBeforeUpdate, onUpdated,
    onBeforeUnmount, onUnmounted } from 'vue';
11
```

```
12 // 1. 替代created/beforeCreate: 组件实例创建后立即执行
13 const count = ref(0);
14 let timer = null;
15 console.log('组件实例创建, count初始值: ', count.value);
16
17 // 2. 挂载前: 模板编译完成, DOM未生成
18 onBeforeMount(() => {
19   console.log('onBeforeMount: DOM未挂载, count值: ', count.value);
20   // 此处无法获取DOM元素
21   const demoEl = document.querySelector('.lifecycle-demo');
22   console.log('挂载前获取DOM: ', demoEl); // null
23 });
24
25 // 3. 挂载后: DOM已生成, 可执行DOM操作
26 onMounted(() => {
27   console.log('onMounted: DOM已挂载');
28   const demoEl = document.querySelector('.lifecycle-demo');
29   demoEl.style.color = '#42b983'; // 修改DOM样式
30
31   // 启动定时器 (模拟业务逻辑)
32   timer = setInterval(() => {
33     count.value++;
34   }, 3000);
35 });
36
37 // 4. 更新前: 数据已变化, DOM未更新
38 onBeforeUpdate((prevVNode) => {
39   console.log('onBeforeUpdate: 数据更新前, 旧DOM状态');
40   console.log('更新前count值: ', count.value);
41   console.log('前一次虚拟DOM: ', prevVNode);
42 });
43
44 // 5. 更新后: DOM已同步数据变化
45 onUpdated((prevVNode) => {
46   console.log('onUpdated: 数据更新后, 新DOM状态');
47   console.log('更新后count值: ', count.value);
48 });
49
50 // 6. 卸载前: 清理资源 (核心! 避免内存泄漏)
51 onBeforeUnmount(() => {
52   console.log('onBeforeUnmount: 组件即将卸载, 清理资源');
53   clearInterval(timer); // 清除定时器
54   timer = null;
55 });
56
57 // 7. 卸载后: 组件实例已销毁
58 onUnmounted(() => {
```

```
59     console.log('onUnmounted: 组件已卸载');
60   });
61 </script>
```

2. 同一钩子的多次调用

组合式API支持对同一生命周期钩子多次调用，Vue会按调用顺序依次执行，这一特性便于拆分不同职责的逻辑（如数据请求、DOM初始化分离）。

Code block

```
1  <script setup>
2  import { onMounted } from 'vue';
3
4  // 逻辑1: 请求用户数据
5  onMounted(async () => {
6    const userData = await fetch('/api/user').then(res => res.json());
7    console.log('用户数据请求完成: ', userData);
8  });
9
10 // 逻辑2: 初始化地图插件
11 onMounted(() => {
12   // 模拟地图初始化
13   console.log('地图插件初始化完成');
14 });
15
16 // 执行顺序: 先请求数据, 后初始化地图
17 </script>
```

四、进阶特性：特殊场景的生命周期钩子

1. keep-alive相关钩子：onActivated与onDeactivated

当组件被 `<keep-alive>` 包裹时，组件卸载时不会被销毁，而是进入缓存状态。此时 `onBeforeUnmount` 和 `onUnmounted` 不会触发，取而代之的是 `onDeactivated`（失活）和 `onActivated`（激活）钩子。

使用示例

Code block

```
1  <!-- 父组件：使用keep-alive包裹子组件 -->
2  <template>
3    <div>
```

```

4      <button @click="showChild = !showChild">切换子组件显示</button>
5      <keep-alive>
6        <ChildComponent v-if="showChild" />
7      </keep-alive>
8    </div>
9  </template>
10
11  <!-- 子组件ChildComponent.vue -->
12  <template>
13    <div>keep-alive子组件: {{ count }}</div>
14  </template>
15
16  <script setup>
17  import { ref, onActivated, onDeactivated } from 'vue';
18
19  const count = ref(0);
20  let timer = null;
21
22  // 组件被激活时触发（首次挂载或从缓存中恢复）
23  onActivated(() => {
24    console.log('onActivated: 组件激活');
25    timer = setInterval(() => {
26      count.value++;
27    }, 1000);
28  });
29
30  // 组件被失活时触发（隐藏但未销毁，进入缓存）
31  onDeactivated(() => {
32    console.log('onDeactivated: 组件失活');
33    clearInterval(timer); // 暂停定时器
34  });
35  </script>

```

2. 错误捕获钩子：onErrorCaptured

该钩子用于捕获子组件（包括所有后代组件）抛出的错误，返回 `true` 可阻止错误继续向上传播，避免程序崩溃。适用于全局错误处理或组件级错误兜底。

Code block

```

1  <script setup>
2  import { onErrorCaptured } from 'vue';
3
4  onErrorCaptured((err, instance, info) => {
5    // err: 错误对象
6    // instance: 抛出错误的子组件实例

```

```
7 // info: 错误来源信息 (如"render"、"mounted")
8 console.error('捕获子组件错误: ', err);
9 console.log('错误组件实例: ', instance);
10 console.log('错误来源: ', info);
11
12 // 显示错误提示 (业务兜底逻辑)
13 alert(`组件出现错误: ${err.message}`);
14
15 // 返回true阻止错误向上传播
16 return true;
17 });
18 </script>
```

五、实战场景：生命周期的典型应用

1. 场景1：挂载后初始化第三方插件

第三方UI插件（如ECharts、地图组件）通常需要绑定DOM元素，必须在 `onMounted` 中初始化。

Code block

```
1 <template>
2   <div id="chart" style="width: 100%; height: 400px;"></div>
3 </template>
4
5 <script setup>
6   import { onMounted, onBeforeUnmount } from 'vue';
7   import * as echarts from 'echarts';
8
9   let myChart = null;
10
11   onMounted(() => {
12     // 初始化ECharts实例 (依赖DOM元素)
13     myChart = echarts.init(document.getElementById('chart'));
14
15     // 配置图表
16     const option = {
17       title: { text: '用户增长趋势' },
18       xAxis: { type: 'category', data: ['1月', '2月', '3月'] },
19       yAxis: { type: 'value' },
20       series: [{ data: [120, 200, 150], type: 'line' }]
21     };
22
23     myChart.setOption(option);
24
25     // 监听窗口resize事件, 适配图表尺寸
```

```
26     window.addEventListener('resize', myChart.resize);
27   });
28
29   onBeforeUnmount(() => {
30     // 销毁图表实例，解绑事件
31     myChart.dispose();
32     window.removeEventListener('resize', myChart.resize);
33   });
34 </script>
```

2. 场景2：数据请求的时机选择

数据请求可根据需求选择不同时机：组件初始化时请求（`setup` 内）、挂载后请求（`onMounted`）、激活时请求（`onActivated`）。

Code block

```
1  <script setup>
2  import { ref, onMounted, onActivated } from 'vue';
3
4  const userList = ref([]);
5
6  // 方式1：组件初始化时请求（适用于数据不依赖DOM，需优先加载）
7  const fetchUserData = async () => {
8    const res = await fetch('/api/user-list');
9    userList.value = await res.json();
10 };
11
12 // 方式2：挂载后请求（适用于请求参数依赖DOM状态）
13 onMounted(() => {
14   // 假设请求参数需要从URL中获取（不依赖DOM，也可在setup内执行）
15   fetchUserData();
16 });
17
18 // 方式3：激活时请求（适用于keep-alive组件，每次显示都刷新数据）
19 onActivated(() => {
20   fetchUserData();
21 });
22 </script>
```

六、核心注意事项与最佳实践

1. 避免在更新钩子中修改数据：`onBeforeUpdate` 和 `onUpdated` 中修改响应式数据会触发无限更新循环，如需修改需添加条件判断。

2. **必须在setup内调用钩子**：生命周期钩子只能在 `<script setup>` 或 `setup()` 函数内调用，不能在异步回调、普通函数中调用（Vue需通过调用时机关联组件实例）。
3. **卸载前必做资源清理**：这是最容易出错的点！在 `onBeforeUnmount` 中必须清理定时器、事件监听、第三方插件实例等，避免内存泄漏。
4. **区分挂载与激活**：被 `keep-alive` 包裹的组件，首次显示触发 `onMounted`，后续显示触发 `onActivated`，需根据业务场景选择钩子。
5. **setup与created的区别**：`setup` 函数中无法访问 `this`（Vue3已移除组合式API中的 `this`），而选项式 `created` 中可通过 `this` 访问组件实例。
6. **错误捕获的边界**：`onErrorCaptured` 只能捕获子组件的错误，无法捕获自身组件的错误（如 `setup`内直接抛出的错误），自身错误需用 `try/catch` 处理。

七、与组件通信的联动：生命周期+插槽+emit

在实际开发中，生命周期函数常与插槽、emit配合实现复杂组件交互。以下是“可编辑表格”组件的综合示例，展示三者的联动逻辑：

Code block

```
1  <!-- 子组件EditableTable.vue -->
2  <template>
3    <table class="editable-table">
4      <thead>
5        <tr><slot name="header">默认表头</slot></tr>
6      </thead>
7      <tbody>
8        <tr v-for="(row, index) in tableData" :key="row.id">
9          <slot name="body" :row="row" :index="index">
10            <td>{{ row.content }}</td>
11          </slot>
12          <td>
13            <button @click="handleEdit(row)">编辑</button>
14          </td>
15        </tr>
16      </tbody>
17    </table>
18  </template>
19
20  <script setup>
21    import { ref, onMounted, defineProps, defineEmits } from 'vue';
22
23    // 接收父组件传递的表格数据 (props)
24    const props = defineProps({
25      initData: { type: Array, required: true }
```



```
26  });
27
28  // 声明要触发的事件 (emit)
29  const emit = defineEmits(['edit-row']);
30
31  // 响应式表格数据
32  const tableData = ref([]);
33
34  // 挂载后初始化表格数据 (生命周期)
35  onMounted(() => {
36    tableData.value = [...props.initData]; // 深拷贝避免修改父组件数据
37  });
38
39  // 编辑按钮点击事件：触发emit通知父组件
40  const handleEdit = (row) => {
41    emit('edit-row', row);
42  };
43  </script>
```

八、总结：组合式API生命周期的核心优势

相比Vue2选项式API，Vue3组合式API的生命周期体系具有以下核心优势：

- **逻辑聚合**：将同一业务相关的生命周期逻辑、数据、方法集中在一起，替代了选项式API中分散的配置，更易维护；
- **灵活复用**：可将生命周期相关逻辑封装为自定义组合函数（如 `useChart`、`useUserData`），跨组件复用；
- **类型友好**：配合TypeScript可实现完整的类型推导，减少类型错误；
- **精简API**：移除冗余的 `created` / `beforeCreate`，通过 `setup` 统一处理初始化逻辑。

掌握生命周期函数的关键在于理解组件的“成长轨迹”，并根据业务场景选择合适的钩子注入逻辑。核心原则是：**初始化逻辑放setup，DOM操作放mounted，资源清理放beforeUnmount**，这是避免常见问题的基础。

（注：文档部分内容可能由 AI 生成）